Restaurant Growth OS

Screen-by-Screen Engineering Blueprint & Junior Developer Guide

Version 1.0 · 27 October 2026 · Proposed implementation blueprint; business decisions marked TBD require owner approval.

1. Purpose and how to use this guide

This guide defines the sequence, proposed frontend screens, backend API contracts, database entities, dependencies, architecture guardrails and acceptance criteria. It is intended to let three developers work in parallel without inventing incompatible patterns. It is also the shared context to pass to Cursor or Claude one screen/task at a time.

- Before coding, read the Architecture & Guardrails JSON and the Screen/API/DB matrix.
- Choose the next screen whose dependencies are complete. Use its screen-specific prompt from cursor-prompts.
- Contract-first: backend and database agree schema and response/error behavior before frontend integration.
- Keep a microtask to roughly 0.5–2 days. Split any task that is larger or touches unrelated modules.
- Every pull request lists changed files, migrations, API/schema changes, tests run and unresolved decisions.
- Do not let AI coding tools invent plan prices, trial lengths, tax/proration policy, entitlement matrix or payment provider behavior.

2. Product and architecture rules

- One multi-tenant product with plan entitlements; do not create separate products per restaurant type.
- Owner growth and evidence-based action tracking is the differentiator; operations modules provide trustworthy source data.
- Start with a modular monolith and well-defined module boundaries; do not start with microservices by default. Extract services only after measured scale/team/availability needs justify it.
- Use GraphQL queries/mutations for application data and GraphQL subscriptions for authorized real-time order/KDS events where appropriate. Use REST for payment webhooks, health endpoints, file upload/download and selected provider integrations.
- PostgreSQL is the transactional source of truth. Redis is optional for measured caching/rate-limit/session coordination needs, not the only store of durable refresh-token/session metadata.
- Separate API request handling from asynchronous workers by code/module and runtime process. Both may initially deploy from one codebase/container image; scale API and worker tasks independently.
- Dockerize from the start. Keep app stateless and externalize state. EC2 and ECS should run the same container image without application-code changes; Terraform describes infrastructure differences.
- Never trust client tenant IDs or hidden navigation. Derive tenant/outlet scope from authenticated identity and authorized membership; enforce in every resolver/service/query.
- Payment domains are separate: internal platform subscription billing versus external restaurant-to-diner payments. Never mix ledgers or provider credentials.
- Mark business metrics measured, estimated, or incomplete. Never treat missing cost data as zero or claim net profit when material costs are missing.
- Use idempotency for order creation, payment actions, billing events and webhook processing. Persist audit trails for sensitive and financial changes.
- All prototype data is illustrative; never copy demo values into production seed data or report outputs without explicit demo labeling.

3. Proposed stack

Layer	Proposal / decision gate
Frontend Web	React + TypeScript + Next.js
Frontend Mobile	React Native + TypeScript; tablet POS decision after hardware/offline proof of concept
Api Backend	TypeScript + NestJS modular monolith with GraphQL driver

Api Styles	GraphQL query/mutation/subscription plus REST for webhooks, health and file transfer
Database	PostgreSQL
Cache Rate Limit	Redis only where required; distributed rate limiting if multiple API replicas
Async	Durable queue + separate worker process; outbox pattern for events that must not be lost
Realtime	GraphQL subscriptions over WebSocket or supported transport; event broker/pub-sub; durable order status history in PostgreSQL
Observability	Structured logs, metrics, traces, error reporting; Sentry or lower-cost alternative evaluated by event volume and retention
Deployment	Docker image; local compose; Terraform; initially low-cost single environment/EC2 if suitable, ECS target; API and workers independently scalable
Storage	Managed object storage for uploads/exports, short-lived signed URLs
Payments	Provider adapter abstraction; separate internal subscription billing and external diner payment integration

4. Three-developer split

Developer A — Frontend

React/Next.js app shell, design tokens, responsive components, auth/onboarding, owner/admin screens, generated GraphQL types, loading/empty/error states.

Developer B — Backend/API

NestJS modules, GraphQL schema/resolvers, domain services, auth/RBAC, subscription resolver, REST webhooks, workers, idempotency and outbox.

Developer C — Database/Platform/Integration

ERD/migrations/indexes, tenant isolation tests, Docker, CI, Terraform baseline, queue/database integration tests, observability, load/security test harness.

All developers share screen IDs and API contracts. Avoid simultaneous edits to the same schema/config files without coordination. One person owns each migration and each GraphQL schema change.

5. Screen sequence and dependencies

The workbook contains 30 screen/workstream rows and 240 microtasks. Screen sequence is a dependency-aware suggested order; P0/P1/P2 are suggested release priorities, not a substitute for final MVP approval.

Seq	ID	Screen	Priority	Depends on
1	S01	Login	P0	S02,S03
2	S02	Sign up	P0	S03,S04
3	S03	Email verification	P0	S01,S02,S04
4	S04	Forgot/reset password	P0	S01
5	S05	Restaurant onboarding	P0	S02,S03
6	S06	Business overview	P0	S05,S07,S08,S09,S10
7	S07	Sales & reports	P0	S05,S08
8	S08	Menu & pricing	P0	S05
9	S09	Orders list/detail	P0	S05,S08
10	S10	POS order builder	P0	S05,S08,S09
11	S11	Inventory & physical counts	P1	S05,S08
12	S12	Purchasing & receiving	P1	S05,S11

13	S13	Expenses	P1	S05,S06
14	S14	Outlet management	P0	S05,S10
15	S15	Growth insights & action tracker	P1	S06,S07,S11,S13
16	S16	Subscription & add-ons	P1	S05,S14
17	S17	Business plan trial request	P1	S02,S05
18	S18	Settings, profile, team and roles	P0	S01,S05
19	S19	Platform overview	P1	S16,S17,S20
20	S20	Restaurant directory	P1	S02,S05,S16
21	S21	Trial request review	P1	S17
22	S22	Subscription management	P1	S16,S20
23	S23	Platform payments & invoices	P1	S16,S22
24	S24	Users & roles	P1	S18,S20
25	S25	System health & alerts	P1	S19
26	S26	Kitchen Display System (KDS)	P1	S09,S10
27	S27	Tables & service status	P2	S10
28	S28	Global states, responsive, accessibility	P0	S01-S27
29	S29	Notifications and delivery preferences	P1	S18,S25
30	S30	Feature entitlement / plan gating	P0	S14,S16,S17

6. API design rules

- Use a versioned schema policy and generated TypeScript types where possible.
- GraphQL field resolvers must not bypass domain services or authorization.
- Apply query depth/complexity limits, pagination, timeouts and rate limits.
- Batch data access to avoid N+1 queries; inspect query plans and add indexes based on evidence.
- Mutations that change financial or operational state validate transitions and accept idempotency keys where retry is possible.
- Subscriptions authorize tenant and outlet at connection/subscription time and re-check event scope; disconnect/re-authenticate expired sessions.
- REST webhooks verify provider signatures, store provider event IDs uniquely, acknowledge safely and process asynchronously where appropriate.
- Use a transactional outbox for DB change + event publication consistency where event loss is unacceptable.

The API_CONTRACT_CATALOG.csv lists proposed REST operations and GraphQL query/mutation/subscription operations. Names are a starting contract, not a claim that these APIs already exist.

7. Data model and multi-tenancy

Core entities include tenants, outlets, users, tenant memberships, scoped role assignments, plans, plan features, subscriptions, subscription items, entitlements, orders, order items, order status events, payments, refunds, menu categories/items/prices, inventory movements/counts, suppliers, purchase orders, expenses, growth insights/actions/outcomes, invoices, billing events, webhook events, sessions, verification challenges, audit events and notification deliveries.

- Every tenant-owned table must have an explicit tenant ownership path. Add tenant_id directly where it improves safe querying and constraints, but ensure it cannot disagree with parent ownership.
- An outlet is normally a child of a tenant. A franchise that is a separate contracting/legal owner may need its own tenant plus an explicit brand/franchise relationship. Confirm with the product owner before schema lock.

- Use foreign keys, unique constraints, check constraints, created/updated timestamps and audit history where appropriate. Add indexes based on expected queries and explain plans.
- Do not store card details. Use payment provider tokens/references and verify provider webhooks.
- Refresh tokens should not be stored in plaintext. Persist hashed token/session metadata, expiry, rotation and revocation in PostgreSQL; Redis is optional for coordination/cache.
- Store order status history durably. GraphQL subscription messages notify clients; query resync recovers after disconnect.

8. Auth, abuse prevention and sessions

- Use OAuth 2.0 / OpenID Connect only after selecting a trusted identity provider and flow; do not invent an OAuth implementation. For first-party login use Authorization Code + PKCE where appropriate.
- Web: prefer Secure, HttpOnly, SameSite cookies for refresh/session secrets where architecture permits. Mobile: use platform secure storage. Never localStorage a long-lived refresh token by default.
- Store refresh-token/session metadata in the database hashed, with rotation, expiry, revocation and reuse detection; Redis can be a cache/coordination layer, not the only durable record.
- Rate-limit by multiple signals: IP/network + account/email/phone + device/session. Use tighter limits for OTP, signup, login and resend; account for shared NATs and avoid IP-only lockouts.
- Use least privilege RBAC plus tenant/outlet scope and feature entitlements. Platform Admin is a separate privilege domain.
- Never log passwords, raw tokens, OTPs, payment credentials or sensitive payloads. Redact crash/telemetry data.
- Run cross-tenant and cross-outlet authorization tests for every data domain.

9. Subscription plans and billing

- Plan catalog, prices, feature entitlements, outlet allowances and add-on prices are configuration, not hard-coded UI constants.
- Starter is intended for one outlet; Growth intended to include three outlets with paid extra outlets. Business requires a trial request and approval with configurable duration/features. Exact pricing/features/trial policy are TBD until product-owner approval.
- Upgrade mid-cycle uses an approved proration policy. Example policy proposed: new plan charge for remaining period minus unused credit for old plan; the user's $30-to-$60 example after 10/30 days results in $40 due. Confirm currency/tax/payment-provider behavior before implementation.
- Downgrade requests are scheduled for next renewal, not applied mid-cycle. Warn if current outlet/feature usage exceeds the target plan and define grace/cleanup behavior.
- Internal SaaS subscription payments and invoices are distinct from external diner payments. Separate tables, provider configuration, permissions and reconciliation.

The requested example—$30 plan upgraded to $60 after 10 days of a 30-day cycle, with $20 unused credit—would produce $40 due under the proposed simple proration formula. This is an example policy, not final billing law; confirm taxes, invoice presentation, currency, billing anchor, provider support and rounding.

10. GraphQL subscriptions and live KDS

- Initial page load uses kitchenOrders query. A subscription delivers orderStatusChanged events after connection.
- Authorize the user, tenant, outlet and station at subscription start; enforce scope when publishing each event.
- Events should carry an order ID, outlet ID, status, version/sequence and timestamp; avoid broadcasting sensitive full order objects.
- Persist order status transitions in PostgreSQL before publishing. Use outbox/worker or equivalent reliable publication where required.
- On disconnect, token expiry or reconnect, re-authenticate and refetch current tickets; subscription delivery alone is not a durable queue.
- Test duplicate events, missed events, reconnect, stale version, unauthorized outlet subscription and tenant isolation.

11. Docker, EC2/ECS, workers and auto-scaling

- Build a stateless API container and a worker container/process from the same repository/image initially. Runtime command/config selects API or worker role.
- Externalize PostgreSQL, queue, object storage, secrets, logs and uploads; do not depend on local container filesystem for durable state.
- Use environment configuration and health/readiness endpoints. Same image should run on EC2 or ECS without application code changes; infrastructure differs in Terraform.
- API and workers scale independently. Set concurrency limits and backpressure; scale workers by queue depth/age, not only CPU.
- Begin with the lowest safe footprint for the validated pilot. 10,000 tenants is not a capacity estimate without outlets, concurrent sessions, order rate, reporting load, retention and availability targets.
- Use CI to build/test/image-scan, publish immutable image, deploy to staging, run migrations safely, health-check and support rollback.

12. Observability, crash reports and alerts

Use structured logs, metrics, traces, server-side error monitoring and supported native/mobile crash reporting. Email and Microsoft Teams notifications should be sent through a durable notification worker with retries, deduplication and delivery status. A client process that crashes may be unable to send a final event, so do not promise guaranteed immediate notification for every crash. Redact tokens, passwords, OTPs, payment data and unnecessary personal information.

13. Screen-by-screen task detail

S01 — Login

Role / platform / priority: Public · Web + Mobile · P0

Dependencies: S02,S03

Frontend scope: Login form, validation, forgot-password link, sign-up link, loading/error states

Proposed APIs: POST /auth/login; POST /auth/refresh; POST /auth/logout; GraphQL me query optional

Database entities: users, auth_sessions, refresh_tokens, audit_events

FE / BE / DB work split: FE: AuthPage, AuthForm, validation, route guards; BE: auth service, session issue/revoke, rate limits; DB: user/session/token metadata

Needs product-owner decision: Authentication provider decision, cookie/mobile token storage decision

Acceptance / safety: No secrets/tokens in logs; generic invalid-credentials response; brute-force protection; tests for lockout/rate limits

S02 — Sign up

Role / platform / priority: Public · Web + Mobile · P0

Dependencies: S03,S04

Frontend scope: Business name, owner name, email, password, terms acceptance

Proposed APIs: POST /auth/signup; POST /verification/email/send; GraphQL signup status optional

Database entities: users, tenants, tenant_memberships, verification_challenges, audit_events

FE / BE / DB work split: FE: signup form; BE: create pending identity and tenant transactionally; DB: unique normalized email, tenant/member foreign keys

Needs product-owner decision: Whether tenant is created before email verification; default plan assignment

Acceptance / safety: Idempotent signup; duplicate email handling; no password plaintext; audit acceptance version

S03 — Email verification

Role / platform / priority: Public · Web + Mobile · P0

Dependencies: S01,S02,S04

Frontend scope: Code/token input, resend cooldown, expiry, success/error, expired state

Proposed APIs: POST /verification/email/confirm; POST /verification/email/resend

Database entities: verification_challenges, users, audit_events

FE / BE / DB work split: FE: code form/countdown; BE: single-use expiring challenge; DB: hashed challenge, expiry, attempts, consumed_at

Needs product-owner decision: Email provider and OTP length/expiry

Acceptance / safety: Attempt limits per IP + account/email + device signals; do not reveal account existence; replay rejected

S04 — Forgot/reset password

Role / platform / priority: Public · Web + Mobile · P0

Dependencies: S01

Frontend scope: Request reset, generic confirmation, reset token page, success/failure

Proposed APIs: POST /auth/password-reset/request; POST /auth/password-reset/confirm

Database entities: password_reset_challenges, users, audit_events

FE / BE / DB work split: FE: request/reset forms; BE: single-use reset flow; DB: hashed token and expiry

Needs product-owner decision: Password policy and email provider

Acceptance / safety: Generic response; short expiry; invalidate sessions after password change; tests for replay/expiry

S05 — Restaurant onboarding

Role / platform / priority: Owner · Web + Mobile · P0

Dependencies: S02,S03

Frontend scope: Restaurant profile, first outlet, city/timezone, currency, outlet count, setup progress

Proposed APIs: GraphQL onboardingStatus; mutation completeOnboarding; mutation createOutlet

Database entities: tenants, outlets, tenant_memberships, tenant_settings, audit_events

FE / BE / DB work split: FE: stepper and form; BE: tenant-scoped mutations; DB: tenant/outlet/settings relations

Needs product-owner decision: Required fields and allowed initial outlet count by plan

Acceptance / safety: Tenant boundary tests; onboarding resumable; audit critical changes

S06 — Business overview

Role / platform / priority: Owner · Web · P0

Dependencies: S05,S07,S08,S09,S10

Frontend scope: Outlet/date filters, KPI cards, sales chart, channel mix, top items, data-quality labels

Proposed APIs: GraphQL ownerOverview, salesTrend, channelMix, topMenuItems; REST export endpoint if needed

Database entities: orders, order_items, payments, outlets, menu_items, refunds, expenses, inventory_counts

FE / BE / DB work split: FE: dashboard layout/chart/loading/empty states; BE: query resolvers + aggregation service; DB: indexes/summary strategy after measurement

Needs product-owner decision: KPI definitions and comparison period

Acceptance / safety: Tenant/outlet scoping on every resolver; mark measured/estimated/incomplete; never claim net profit with missing costs

S07 — Sales & reports

Role / platform / priority: Owner · Web · P0

Dependencies: S05,S08

Frontend scope: Date/outlet/channel filters, sales/refund/net sales/orders/AOV, daily summary, export

Proposed APIs: GraphQL salesReport, salesTrend; REST POST /reports/exports and GET /reports/exports/:id

Database entities: orders, order_items, payments, refunds, outlets, report_exports

FE / BE / DB work split: FE: filterable report; BE: KPI service and async export worker; DB: reporting indexes/materialized views only if justified

Needs product-owner decision: Approved sales/refund/tax definitions

Acceptance / safety: Consistent timezone/currency; totals reconcile to order/payment ledger; large export async

S08 — Menu & pricing

Role / platform / priority: Owner/Manager · Web + Tablet · P0

Dependencies: S05

Frontend scope: Categories, menu items, price, tax category, availability, modifiers if approved

Proposed APIs: GraphQL menuCategories, menuItems; mutations create/update/archiveMenuItem, setItemAvailability

Database entities: menu_categories, menu_items, menu_item_prices, tax_categories, audit_events

FE / BE / DB work split: FE: list/card/form; BE: menu service and authorization; DB: tenant/outlet scope and price history if required

Needs product-owner decision: Menu shared across outlets or outlet overrides; tax rules

Acceptance / safety: Money stored in integer minor units or agreed decimal model; audit price changes; validation tests

S09 — Orders list/detail

Role / platform / priority: Owner/Staff · Web + Tablet · P0

Dependencies: S05,S08

Frontend scope: Search/filter orders, status, detail modal/page, timeline, refund visibility

Proposed APIs: GraphQL orders, orderById; mutations cancelOrder/requestRefund only if permissioned; REST webhook routes for providers

Database entities: orders, order_items, order_status_events, payments, refunds, outlets, users

FE / BE / DB work split: FE: list/detail/timeline; BE: order query/service, explicit transition rules; DB: order + immutable event history

Needs product-owner decision: Order state machine and cancel/refund policy

Acceptance / safety: Idempotency for writes; server-side transition validation; payment status reconciled from provider

S10 — POS order builder

Role / platform / priority: Staff · Tablet first; Web fallback · P0

Dependencies: S05,S08,S09

Frontend scope: Category filter, item select, quantities, modifiers, cart, customer/order type, totals, payment handoff

Proposed APIs: GraphQL posMenu, mutation createOrder, mutation updateDraftOrder; REST payment intent/webhook endpoints; optional subscription orderUpdated

Database entities: orders, order_items, menu_items, order_status_events, payments, idempotency_keys

FE / BE / DB work split: FE: tablet-first cart; BE: transactional order creation; DB: idempotency unique key, order number sequence strategy

Needs product-owner decision: Hardware, offline scope, taxes, payment integration

Acceptance / safety: Do not store card data; server recalculates totals; duplicate-submit tests; offline only if explicitly designed

S11 — Inventory & physical counts

Role / platform / priority: Owner/Manager · Web + Tablet · P1

Dependencies: S05,S08

Frontend scope: Ingredient list, on-hand movement ledger, reorder threshold, physical count and variance review

Proposed APIs: GraphQL ingredients, stockBalances, stockMovements, createPhysicalCount, finalizePhysicalCount

Database entities: ingredients, stock_movements, stock_counts, stock_count_lines, suppliers, audit_events

FE / BE / DB work split: FE: inventory table and count wizard; BE: stock ledger/count workflow; DB: append-only movements plus count records

Needs product-owner decision: Units of measure and recipe support scope

Acceptance / safety: Theoretical recipe consumption != actual stock; variance needs count + reconciliation; prevent negative/invalid movements by policy

S12 — Purchasing & receiving

Role / platform / priority: Owner/Manager · Web · P1

Dependencies: S05,S11

Frontend scope: Suppliers, purchase orders, line items, expected delivery, receiving and invoice status

Proposed APIs: GraphQL suppliers, purchaseOrders; mutations createPurchaseOrder, receivePurchaseOrder; REST import endpoint optional

Database entities: suppliers, purchase_orders, purchase_order_lines, stock_movements, supplier_invoices

FE / BE / DB work split: FE: PO form/list/detail; BE: PO state machine and receive transaction; DB: link receiving to stock ledger

Needs product-owner decision: Partial receiving and supplier invoice matching

Acceptance / safety: Receiving idempotent; audit adjustments; do not double-count invoice and stock receipt

S13 — Expenses

Role / platform / priority: Owner/Manager · Web · P1

Dependencies: S05,S06

Frontend scope: Expense category, vendor, amount, date, outlet, approval state, attachment optional

Proposed APIs: GraphQL expenses; mutations createExpense, updateExpense, approveExpense; REST attachment upload

Database entities: expenses, expense_categories, files, audit_events

FE / BE / DB work split: FE: form/list/filters; BE: validation, approval authorization; DB: tenant/outlet ownership

Needs product-owner decision: Expense approval rules and attachment storage

Acceptance / safety: Net profit incomplete until material costs are configured; file type/size checks

S14 — Outlet management

Role / platform / priority: Owner · Web · P0

Dependencies: S05,S10

Frontend scope: Outlet list, create/edit/disable, manager assignment, plan allowance and outlet switcher

Proposed APIs: GraphQL outlets; mutations createOutlet, updateOutlet, deactivateOutlet, assignOutletRole

Database entities: outlets, tenant_memberships, role_assignments, plan_subscriptions, audit_events

FE / BE / DB work split: FE: outlet cards/list and switcher; BE: plan entitlement check; DB: tenant-scoped outlet records

Needs product-owner decision: Whether Growth includes 3 outlets and approved extra-outlet price

Acceptance / safety: Tenant admin cannot access other tenants; cannot exceed entitlement without approved add-on

S15 — Growth insights & action tracker

Role / platform / priority: Owner/Manager · Web · P1

Dependencies: S06,S07,S11,S13

Frontend scope: Evidence, confidence, data limitations, approve/reject, owner, baseline, target, due date, outcome

Proposed APIs: GraphQL growthInsights, growthActions; mutations approveGrowthAction, updateAction, recordOutcome

Database entities: growth_insights, growth_actions, action_outcomes, audit_events

FE / BE / DB work split: FE: insight cards/detail/action tracker; BE: explainable rules and action workflow; DB: store evidence references and outcome snapshots

Needs product-owner decision: Initial deterministic recommendation rules and KPI baselines

Acceptance / safety: No unsupported causality; owner approval before execution; capture outcome and baseline

S16 — Subscription & add-ons

Role / platform / priority: Owner · Web · P1

Dependencies: S05,S14

Frontend scope: Current plan, feature matrix, outlet allowance, add-ons, invoices, upgrade preview, downgrade at renewal

Proposed APIs: GraphQL currentSubscription, planCatalog, invoiceHistory, upgradePreview; mutations requestUpgrade, scheduleDowngrade, cancelScheduledChange

Database entities: plans, plan_features, plan_subscriptions, subscription_items, invoices, billing_events, tenant_entitlements

FE / BE / DB work split: FE: plan cards/compare/confirmation; BE: billing ledger/proration and entitlement resolver; DB: effective-dated subscription changes

Needs product-owner decision: Final pricing, trial policy, proration/tax/currency policy

Acceptance / safety: Upgrade charge = target-period charge minus unused current-plan credit under approved policy; downgrade only effective at renewal; immutable invoice ledger

S17 — Business plan trial request

Role / platform / priority: Public/Owner · Web · P1

Dependencies: S02,S05

Frontend scope: Trial request form, status, admin decision, duration and extension visibility

Proposed APIs: GraphQL trialRequestStatus; mutation requestBusinessTrial; admin mutation reviewTrialRequest

Database entities: trial_requests, plan_subscriptions, audit_events, tenant_entitlements

FE / BE / DB work split: FE: request/status views; BE: approval workflow and entitlement grants; DB: approved-by, timestamps, duration

Needs product-owner decision: Trial duration, feature set, extension rules

Acceptance / safety: No Business access before approval; extension audited; trial expiry enforced server-side

S18 — Settings, profile, team and roles

Role / platform / priority: Owner · Web + Mobile · P0

Dependencies: S01,S05

Frontend scope: Profile, password/security, team invite, roles, outlet scope, notifications

Proposed APIs: GraphQL me, members, roleCatalog; mutations updateProfile, inviteMember, updateMemberRole, revokeSessions; REST invite/verification email

Database entities: users, tenant_memberships, role_assignments, auth_sessions, notification_preferences, audit_events

FE / BE / DB work split: FE: settings tabs and member table; BE: RBAC/tenant scope; DB: membership and scoped role grants

Needs product-owner decision: Exact role matrix and invite acceptance

Acceptance / safety: Least privilege; owner cannot grant platform admin; session revoke works immediately

S19 — Platform overview

Role / platform / priority: Platform Admin · Web · P1

Dependencies: S16,S17,S20

Frontend scope: Tenant counts, subscriptions, trial queue, payment exceptions, system health summaries

Proposed APIs: GraphQL platformOverview guarded by platform-admin policy

Database entities: tenants, plan_subscriptions, trial_requests, billing_events, operational_metrics

FE / BE / DB work split: FE: admin dashboard; BE: separate privileged policy and aggregates; DB: operational read strategy

Needs product-owner decision: Admin KPI definitions

Acceptance / safety: Platform admin access must be separate from tenant roles and audited

S20 — Restaurant directory

Role / platform / priority: Platform Admin · Web · P1

Dependencies: S02,S05,S16

Frontend scope: Search tenant, plan/outlets/status, suspend/reactivate with reason, details

Proposed APIs: GraphQL platformTenants, tenantById; mutations suspendTenant, reactivateTenant

Database entities: tenants, outlets, plan_subscriptions, audit_events

FE / BE / DB work split: FE: searchable directory/detail; BE: privileged service; DB: tenant status/audit

Needs product-owner decision: Suspension policy and owner notification

Acceptance / safety: Every privileged change has actor, reason, timestamp; never expose secrets

S21 — Trial request review

Role / platform / priority: Platform Admin · Web · P1

Dependencies: S17

Frontend scope: Review request, approve/reject, configure approved duration/features, extension audit

Proposed APIs: GraphQL platformTrialRequests; mutations approveTrial, rejectTrial, extendTrial

Database entities: trial_requests, plan_subscriptions, tenant_entitlements, audit_events

FE / BE / DB work split: FE: review queue/detail; BE: workflow validation; DB: decision history

Needs product-owner decision: Trial durations and extension limits

Acceptance / safety: Approval is transactional; expiry job revokes trial entitlements correctly

S22 — Subscription management

Role / platform / priority: Platform Admin · Web · P1

Dependencies: S16,S20

Frontend scope: Search subscription, current/effective plan, renewal, scheduled changes, exceptions

Proposed APIs: GraphQL platformSubscriptions; mutation correctSubscription with reason and audit

Database entities: plan_subscriptions, subscription_items, invoices, billing_events, tenant_entitlements

FE / BE / DB work split: FE: detail/history; BE: admin correction workflow; DB: effective-dated history

Needs product-owner decision: Allowed manual correction policy

Acceptance / safety: No silent mutation of invoice history; corrective entries auditable

S23 — Platform payments & invoices

Role / platform / priority: Platform Admin · Web · P1

Dependencies: S16,S22

Frontend scope: Internal subscription payments, invoice view, failed payments, reconciliation

Proposed APIs: GraphQL platformInvoices, billingEvents; REST payment-provider webhooks and reconciliation job

Database entities: invoices, billing_events, payment_attempts, provider_webhook_events

FE / BE / DB work split: FE: billing table/details; BE: provider adapter/webhook idempotency; DB: immutable event ledger

Needs product-owner decision: Payment gateway and supported countries/currencies

Acceptance / safety: Separate internal SaaS subscription billing from restaurant customer payment processing; verify webhook signatures

S24 — Users & roles

Role / platform / priority: Platform Admin · Web · P1

Dependencies: S18,S20

Frontend scope: Platform users, tenant memberships, role/scope, invite/revoke

Proposed APIs: GraphQL platformUsers; mutations platformInviteUser, disableUser, revokeUserSessions

Database entities: users, tenant_memberships, role_assignments, auth_sessions, audit_events

FE / BE / DB work split: FE: user directory; BE: elevated policy; DB: membership and role audit

Needs product-owner decision: Support impersonation? default no

Acceptance / safety: Never implement silent impersonation; if later approved, strong banner, reason, audit and time limit

S25 — System health & alerts

Role / platform / priority: Platform Admin · Web · P1

Dependencies: S19

Frontend scope: API/worker health, queue backlog, webhook failures, notification delivery, incident details

Proposed APIs: GraphQL operationalSummary; REST health endpoints; worker metrics endpoint

Database entities: operational_events, notification_deliveries, job_runs, audit_events

FE / BE / DB work split: FE: status dashboard; BE: telemetry aggregation; DB: event metadata not raw secrets

Needs product-owner decision: Sentry/alternative, email and Teams provider

Acceptance / safety: Redact PII/secrets; alert delivery itself monitored; do not show fabricated live values

S26 — Kitchen Display System (KDS)

Role / platform / priority: Staff · Tablet/Display · P1

Dependencies: S09,S10

Frontend scope: Live tickets, new/preparing/ready/completed, station filters, reconnect/resync

Proposed APIs: GraphQL query kitchenOrders; GraphQL subscription orderStatusChanged; mutation advanceKitchenStatus

Database entities: orders, order_items, order_status_events, outlet_memberships, subscription event broker

FE / BE / DB work split: FE: ticket board and state transitions; BE: subscription auth, event publication; DB: durable order event history

Needs product-owner decision: WebSocket provider/transport and event retention

Acceptance / safety: Subscription events are hints; refetch/resync after reconnect; tenant/outlet authorization on subscribe and every event

S27 — Tables & service status

Role / platform / priority: Staff · Tablet · P2

Dependencies: S10

Frontend scope: Table grid, capacity, occupied/available/reserved, open order linkage

Proposed APIs: GraphQL tables; mutations createTable, updateTableStatus, assignOrderToTable

Database entities: restaurant_tables, orders, order_status_events

FE / BE / DB work split: FE: tablet grid; BE: state transitions; DB: tenant/outlet-scoped tables

Needs product-owner decision: Confirm table service in MVP

Acceptance / safety: Only implement if approved; prevent conflicting table assignments

S28 — Global states, responsive, accessibility

Role / platform / priority: All roles · All · P0

Dependencies: S01-S27

Frontend scope: Loading, empty, error, permission denied, no-plan feature, offline if approved, mobile/tablet layouts

Proposed APIs: No new business API; reuse typed API errors and feature-entitlement queries

Database entities: No new core tables; audit accessibility exceptions only if needed

FE / BE / DB work split: FE: reusable state components, responsive QA, keyboard/focus, contrast, touch targets; BE: consistent errors

Needs product-owner decision: Offline support scope and supported browser/device matrix

Acceptance / safety: Every screen has loading/error/empty/success; no client-only authorization; tests at breakpoints

S29 — Notifications and delivery preferences

Role / platform / priority: All roles · Web + Mobile · P1

Dependencies: S18,S25

Frontend scope: Email/Teams platform alert configuration; user notification preferences; delivery history where allowed

Proposed APIs: GraphQL notificationPreferences; mutation updateNotificationPreferences; worker sends email/Teams; REST provider callbacks if supported

Database entities: notification_preferences, notification_deliveries, job_runs, operational_events

FE / BE / DB work split: FE: settings and delivery status; BE: outbox/worker/retry; DB: delivery attempts and dedupe key

Needs product-owner decision: Email/Teams providers and event severity rules

Acceptance / safety: Retry with backoff, dedupe alerts, redact details; crash reporting is not equivalent to guaranteed client crash capture

S30 — Feature entitlement / plan gating

Role / platform / priority: All roles · All · P0

Dependencies: S14,S16,S17

Frontend scope: Hide/disable gated features with explanatory upgrade or trial states

Proposed APIs: GraphQL myEntitlements; shared server policy used by every resolver/mutation

Database entities: plans, plan_features, plan_subscriptions, subscription_items, tenant_entitlements

FE / BE / DB work split: FE: entitlement-aware navigation and guard; BE: centralized authorization policy; DB: effective dates and overrides

Needs product-owner decision: Starter/Growth/Business exact feature matrix and add-on prices

Acceptance / safety: Server always enforces entitlements; UI hiding alone is not security

14. Definition of done

- Approved design and API contract exist; generated/shared types are current.
- Loading, empty, validation, error, success, permission-denied and plan-gated states are implemented as applicable.
- Tenant/outlet authorization is tested on the server for reads, writes and subscriptions.
- Mutations that affect money/order state are transactional, validated and idempotent where retries are possible.
- Database migration has constraints/indexes and has been tested against a clean database.
- Unit and integration tests run; key flows have e2e coverage; typecheck/lint pass.
- Logs and alerts are useful and redact sensitive data; retries and failure modes are tested.
- Documentation and tracker updated with test evidence and unresolved decisions.

15. Open decisions — do not guess

- Approve exact MVP screen scope and role matrix.
- Approve exact Starter/Growth/Business feature matrix, prices, extra-outlet price, trial duration and extension policy.
- Select identity provider and email/OTP provider.
- Select payment gateway(s) for platform subscription billing and external diner payments.
- Confirm tablet/POS hardware, offline behavior, printers, cash drawer and payment terminal needs.
- Select queue/broker and GraphQL subscription transport.
- Choose Sentry versus alternatives after projected event volume, retention and alerting requirements.
- Confirm whether table management is in MVP.
- Confirm currency/tax/invoice rules and jurisdiction requirements.
- Set load-test assumptions: tenants, outlets per tenant, concurrent users, order write rate, report volume, retention and availability targets.

16. How to use Cursor or Claude safely

- Paste only one screen-specific prompt at a time, after confirming its dependencies.
- Ask the tool to inspect the repository and report the proposed file plan before edits.
- Reject broad refactors, new services, new product features or unapproved pricing changes.
- Review diff and tests. Never accept 'tests pass' without actual command output.
- Commit small slices and keep the workbook status current.